



ITI · AI AGENTS DEVELOPMENT USING PYTHON TRACK

Python Programming for AI Applications

Eng. Fatma Elraey*Day 6 — Introduction to pandas (Working with Tables)*

Field	Detail
Instructor	Fatma Elraey
Track	Python for AI Applications (6-Day Course)
Session	Day 6 of 6 (Final Day)
Focus	A gentle, hands-on introduction to pandas DataFrames — the labeled-table tool almost every real dataset passes through before it ever reaches a machine learning model.

Quick Recap — Day 5

On Day 5 you met the NumPy array — a fast, uniform grid of numbers. Today, on our final day, we meet pandas, a library built directly on top of NumPy that adds row and column labels, mixed data types, and a huge toolbox for exploring, cleaning, and summarizing real-world data. If arrays are the engine, pandas is the dashboard you'll actually spend most of your time using.

Day 6 Outline

1. Why pandas? From Arrays to Tables
2. Creating & Exploring a DataFrame
3. Selecting Rows & Columns
4. Filtering & Sorting Data
5. New Columns & Missing Data
6. Grouping & Summarizing

01

Why pandas? From Arrays to Tables

A NumPy array is great for numbers, but real data is rarely just numbers — it has names, dates, categories, and column labels you want to keep attached to the data. pandas gives you two new types built for exactly that: the Series (a single labeled column) and the DataFrame (a full labeled table, like a spreadsheet).

Like NumPy, pandas needs to be imported first. Almost everyone uses the nickname `pd`:

```
import pandas as pd
```

NumPy array*just numbers*

90	85
70	88
82	77

**pandas DataFrame***numbers + row & column labels*

	quiz1	quiz2
Amir	90	85
Sara	70	88
Leo	82	77

Figure 6.1 — A DataFrame is the same kind of data as an array, plus row and column labels.

Here is the smallest possible Series — a single labeled column of data:

```
scores = pd.Series([90, 85, 78], index=["Amir", "Sara", "Leo"])
print(scores)
# Amir    90
# Sara     85
# Leo      78
# dtype: int64
```

Notice each value now has a name attached to it, instead of just a position. That one idea — giving data meaningful labels — is what pandas is built around.

AI CONNECTION Before any dataset reaches scikit-learn or a neural network, someone almost always loaded it, cleaned it, and explored it using pandas first. It's the tool that sits between "a messy CSV file" and "a NumPy array ready for a model."



✓ QUICK CHECK — Why pandas? From Arrays to Tables

Q1. What are the two main pandas data types introduced here?

Answer: *Series (a single labeled column) and DataFrame (a full labeled table).*

Q2. What does a pandas Series add on top of a plain list of numbers?

Answer: *A label (index) attached to every value, instead of just its position.*

02

Creating & Exploring a DataFrame

The easiest way to build a DataFrame is from a dictionary, where each key becomes a column:

```
data = {
    "name": ["Amir", "Sara", "Leo"],
    "quiz1": [90, 70, 82],
    "quiz2": [85, 60, 77],
}
df = pd.DataFrame(data)
print(df)
#   name  quiz1  quiz2
# 0  Amir     90     85
# 1  Sara     70     60
# 2   Leo     82     77
```

	name	quiz1	quiz2	pass
0	Amir	90	85	True
1	Sara	70	60	False
2	Leo	82	77	True

Figure 6.2 — Every DataFrame has an index (row labels), columns, and values.

Once you have a DataFrame, a handful of methods let you get a quick feel for it before doing anything else:

```
print(df.shape)      # (3, 3) -- 3 rows, 3 columns
print(df.columns)    # Index(['name', 'quiz1', 'quiz2'])
print(df.dtypes)     # what type each column holds
print(df.head(2))    # just the first 2 rows
print(df.describe()) # quick stats for numeric columns
```

`df.describe()` is especially useful the moment you load any new dataset — in one line it shows you the count, mean, min, max, and more for every numeric column, giving you an instant sanity check on the data.

AI CONNECTION The very first thing done with almost any real dataset — before training a single model — is `df.head()`, `df.shape`, and `df.describe()`. It's the quickest way to catch a wrong column, an unexpected data type, or a suspicious value before it causes problems later.



✓ QUICK CHECK — Creating & Exploring a DataFrame

Q1. What does each key in a dictionary become when you build a DataFrame from it?

Answer: *A column of the DataFrame.*

Q2. What does `df.describe()` show you?

Answer: *Quick summary statistics (count, mean, min, max, and more) for every numeric column.*

03

Selecting Rows & Columns

Selecting a single column returns a Series, using square brackets with the column name:

```
print(df["quiz1"])
# 0    90
# 1    70
# 2    82
# Name: quiz1, dtype: int64

print(df[["name", "quiz1"]]) # two columns, still a DataFrame
```

For rows, pandas gives you two selection tools that look similar but mean different things: `.loc` selects by label, and `.iloc` selects by position, just like a list index.

0	Amir	90	df.loc["Sara"] → by label
1	Sara	70	df.iloc[1] → by position
2	Leo	82	

Figure 6.3 — `.loc` looks up a label; `.iloc` looks up a position.

If we set the student names as the index, the difference becomes clear:

```
df2 = df.set_index("name")
print(df2)
#      quiz1  quiz2
# name
# Amir     90     85
# Sara      70     60
# Leo       82     77

print(df2.loc["Sara"]) # row with the label "Sara"
print(df2.iloc[1])     # row at position 1 -- same result here
```

`.loc` also accepts a row label and a column name together, just like a 2D NumPy array with a comma:

```
print(df2.loc["Sara", "quiz1"]) # 70
```



AI CONNECTION *.loc and .iloc are everywhere once you start cleaning real datasets — .iloc is handy for "the first 100 rows" while .loc is handy for "the row where the ID is X," and knowing which one you need avoids a lot of confusing bugs.*

✓ QUICK CHECK — Selecting Rows & Columns

Q1. What does `df["quiz1"]` return?

Answer: *A Series — the single quiz1 column, with its labels.*

Q2. What is the difference between `.loc` and `.iloc`?

Answer: *.loc selects by label (name), while .iloc selects by numeric position, like a list index.*

04

Filtering & Sorting Data

Just like in NumPy, you can filter a DataFrame using a plain comparison — pandas keeps only the rows where the condition is true:

```
passed = df[df["quiz1"] >= 80]
print(passed)
#   name  quiz1  quiz2
# 0  Amir    90     85
# 2   Leo     82     77
```

df[df["score"] >= 60]

Amir	90	kept
Sara	55	dropped
Leo	82	kept
Nina	40	dropped

Figure 6.4 — Rows where the condition is True are kept; the rest are dropped.

You can also combine more than one condition, using & (and) or | (or) — each condition needs its own parentheses:

```
both = df[(df["quiz1"] >= 80) & (df["quiz2"] >= 80)]
print(both)
#   name  quiz1  quiz2
# 0  Amir    90     85
```

Sorting is one method call — `sort_values()` takes the column name to sort by:

```
print(df.sort_values("quiz1")) # smallest quiz1 first
print(df.sort_values("quiz1", ascending=False)) # largest first
```

AI CONNECTION Filtering rows like "only rows where the label is missing" or "only rows from the last 30 days" is one of the most common operations in preparing a dataset — you'll use this `>= / & / |` pattern constantly.



✓ QUICK CHECK — Filtering & Sorting Data

Q1. What does `df[df["quiz1"] >= 80]` do?

Answer: Keeps only the rows where the quiz1 column is 80 or higher.

Q2. How do you combine two filter conditions with 'and'?

Answer: Wrap each condition in parentheses and join them with `&` — e.g. `df[(cond1) & (cond2)]`.

05

New Columns & Missing Data

Adding a new column is as simple as assigning to a new column name — and just like NumPy, the math applies to every row at once:

```
df["average"] = (df["quiz1"] + df["quiz2"]) / 2
print(df)
#   name  quiz1  quiz2  average
# 0  Amir    90    85    87.5
# 1  Sara    70    60    65.0
# 2   Leo    82    77    79.5
```

A common next step is turning a numeric column into a True/False (or category) column with a condition:

```
df["passed"] = df["average"] >= 70
print(df[["name", "average", "passed"]])
#   name  average  passed
# 0  Amir    87.5    True
# 1  Sara    65.0   False
# 2   Leo    79.5    True
```

Real data is often incomplete. A missing value shows up as NaN ("Not a Number"), and pandas has simple tools for finding and dealing with it:

```
import numpy as np
df.loc[1, "quiz2"] = np.nan # pretend Sara's quiz2 is missing

print(df.isna())           # True/False grid of missing spots
print(df["quiz2"].isna().sum()) # 1 -- how many are missing

filled = df.fillna(0)       # replace missing values with 0
dropped = df.dropna()       # or remove rows with any missing value
```

AI CONNECTION Missing data is one of the very first problems every real dataset throws at you, and a model usually can't train on NaN values directly — deciding whether to `fillna()` or `dropna()` (and with what) is a genuine early decision in almost every AI project.

✓ QUICK CHECK — New Columns & Missing Data

Q1. How do you add a new column that's the sum of two existing columns?

Answer: `df["new_col"] = df["col_a"] + df["col_b"]` — assign to a new column name.

Q2. What are the two basic ways to handle missing (NaN) values shown here?

Answer: `fillna(value)` to replace missing values, or `dropna()` to remove rows that have any missing value.

06

Grouping & Summarizing

`groupby()` answers questions like "what's the average score per group?" It works in three steps: split the rows into groups, apply a calculation to each group, then combine the results into one small summary table.

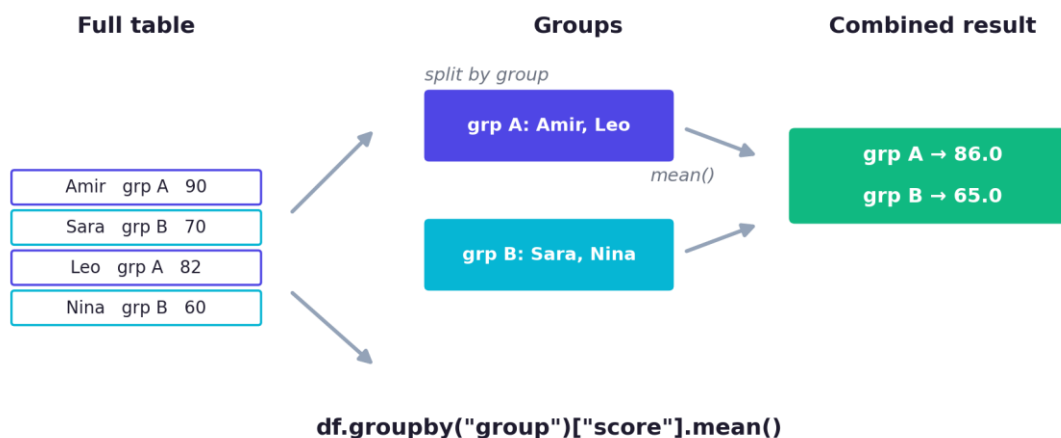


Figure 6.5 — Split the rows by group, apply a calculation, combine into a summary.

```
data = {
    "name": ["Amir", "Sara", "Leo", "Nina"],
    "group": ["A", "B", "A", "B"],
    "score": [90, 70, 82, 60],
}
df = pd.DataFrame(data)

print(df.groupby("group")["score"].mean())
# group
# A      86.0
# B      65.0
# Name: score, dtype: float64
```

`groupby()` works with other aggregations too, and you can ask for more than one at once with `.agg()`:

```
print(df.groupby("group")["score"].count()) # how many per group
print(df.groupby("group")["score"].max())   # best score per group
print(df.groupby("group")["score"].agg(["mean", "max", "count"]))
```

That's a wrap for the whole track! In six days you've gone from basic Python syntax, through functions and files, to classes, NumPy arrays, and now pandas tables — the exact toolbox used to load, clean, and explore data before it ever reaches a machine learning model.

AI CONNECTION Reporting "accuracy per class" or "average error per category" after evaluating a model is almost always one `groupby()` call away — this single method is one of the most-used tools in any real data analysis or model evaluation workflow.

✓ QUICK CHECK — Grouping & Summarizing

Q1. What are the three steps `groupby()` conceptually performs?

Answer: Split the rows into groups, apply a calculation to each group, then combine the results into a summary.

Q2. How would you get both the mean and the max score per group in one call?

Answer: `df.groupby("group")["score"].agg(["mean", "max"])`



Practical Time

Work through these on your own before checking the marked answer.

Practice Question 1

What will this code print?

```
df = pd.DataFrame({"a": [1, 2, 3]})  
print(type(df["a"]))
```

- A. <class 'list'>
- B. <class 'pandas.core.series.Series'> ✓
- C. <class 'pandas.core.frame.DataFrame'>
- D. <class 'dict'>

Why: Selecting a single column with [] returns a Series, not a DataFrame — a DataFrame is a collection of Series sharing one index.

Practice Question 2

What will this code print?

```
df = pd.DataFrame({"score": [40, 90, 60]})  
print(df[df["score"] >= 60])
```

- A. A DataFrame with rows 1 and 2 only ✓
- B. [True, True, True]
- C. A DataFrame with all 3 rows
- D. KeyError

Why: df["score"] >= 60 builds a True/False Series, and using it inside [] keeps only the rows where it's True (score 90 and 60).

Practice Question 3

For a DataFrame df2 indexed by student name, what does df2.loc["Sara"] return, compared to df2.iloc[1]?

- A. They always return different rows
- B. loc looks up by label, iloc by position — they can return the same row ✓**
- C. iloc only works on columns, not rows
- D. loc raises an error on a named index

Why: .loc selects by the row's label ("Sara"), while .iloc selects by numeric position (1) — if Sara happens to be at position 1, both return the same row.

Practice Question 4

What will this code print?

```
df = pd.DataFrame({"x": [1, 2, 3]})  
df["y"] = df["x"] * 10  
print(df["y"].tolist())
```

- A. [1, 2, 3]
- B. [10, 20, 30] ✓**
- C. TypeError
- D. [1, 2, 3, 10, 20, 30]

Why: Assigning to a new column name creates that column, and the math (10) applies to every row at once, just like a NumPy array.*

Practice Question 5

A column has 2 missing (NaN) values out of 5 rows. What does df.dropna() do to that column's rows?

- A. Replaces the NaN values with 0
- B. Removes the 2 rows that contain a NaN, leaving 3 rows ✓**
- C. Does nothing unless a column name is given
- D. Deletes the entire column

Why: dropna() removes whole rows that contain at least one missing value, by default — here that's the 2 rows with NaN, leaving 3.



Practice Question 6

What does `df.groupby("team")["points"].mean()` return?

- A. The overall average of the points column
- B. One average points value per unique team ✓**
- C. A single True/False value
- D. The team with the highest points

Why: `groupby("team")` splits the rows into one group per unique team value, and `["points"].mean()` computes the average points within each group separately.